

中山大学网络安全学院必修课程

程序设计I

C Programming

岳 晟

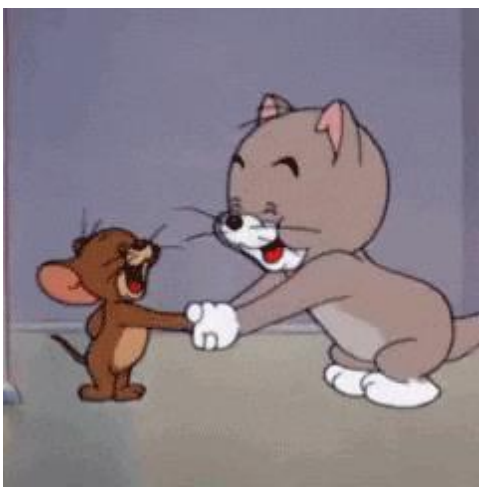
网络安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季

联系方式

- 教师：岳晟
- 邮箱：yuesh5@mail.sysu.edu.cn
- 助教：龙语嫣（硕士二年级），吴劭雄（硕士二年级）
- 课程群：



课程须知

□ 学习资料

- 课件

- Stephen Prata. C Primer Plus (第六版) 中文版.

- 谭浩强. C程序设计 (第五版).

□ 使用  雨课堂 [查看课程回放](#)

课程须知

□ 考核标准 (**理论课和实验课成绩相同**)

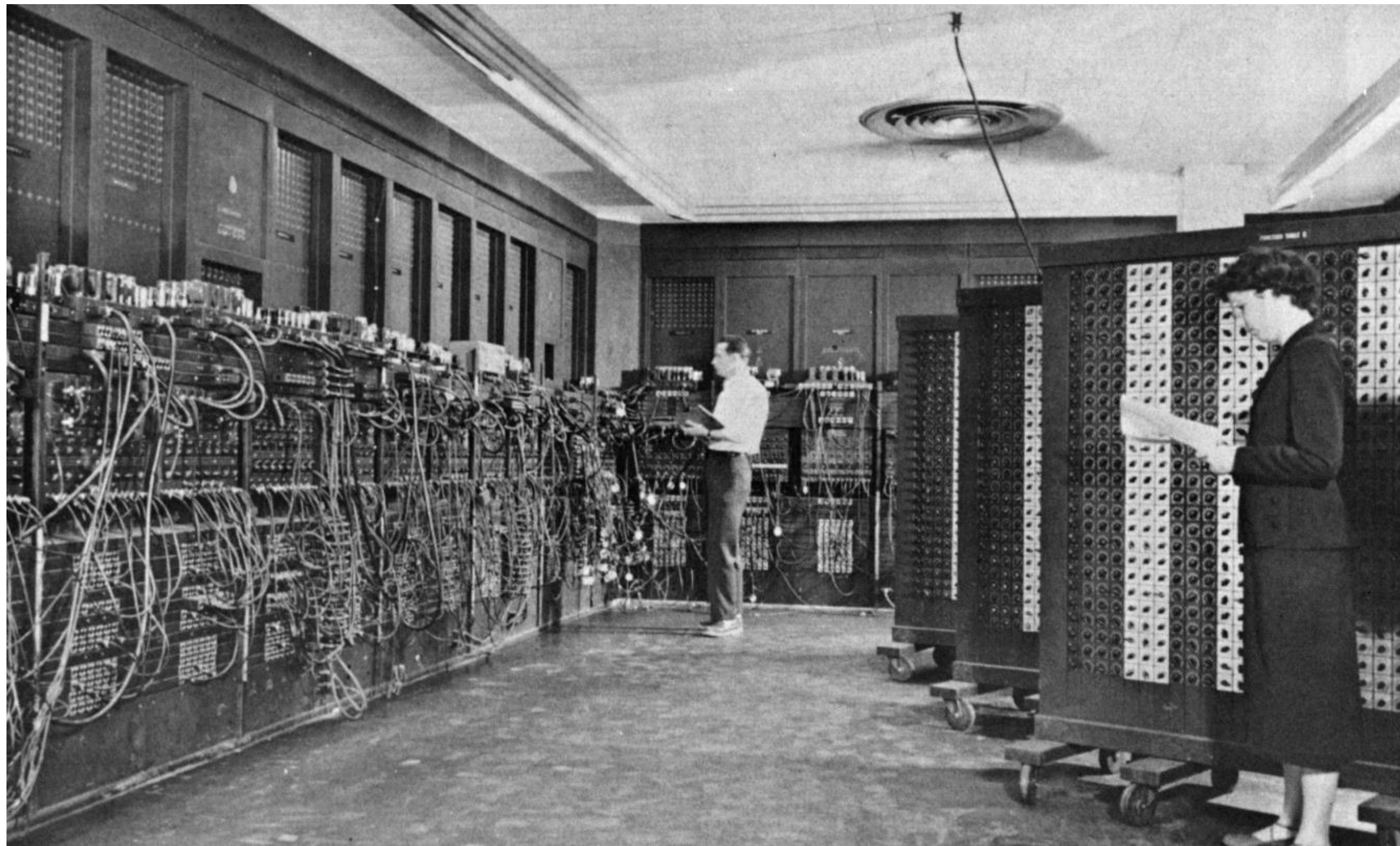
- 总评成绩 = 平时成绩*40% + 期末成绩*60%
- 平时成绩 = 上机实验与测验*80% + 课堂表现与考勤*20%
- 期末成绩 = 期末理论考试成绩*40% + 期末实验考试成绩*60%

□ 期末考试

- 形式：机试
- 时间（暂定）：1月13日（周二） 9:30-11:30、14:30-16:30

本课程的目标

- 本课程主要目标是通过学习程序设计和编程语言，来**理解计算机是如何解决问题的**，理解计算机的能与不能，擅长与非擅长



中山大学专业课《程序设计I》

第一节 C语言概述

C Programming

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季

大模型都能写代码了，为什么我们还要学编程？

程序員處「食物鏈」底層 會被AI取代？

知·析趨勢

發布時間：2024/07/11

A+ A- 關注文章 儲存文章

分享： f 微博 邮件 链接

21 21财经

打开APP >

月入几万的程序员，要被AI取代了？

光锥智能 南财号

+ 关注

SOFTWARE DEVELOPMENT

Coding Sucks Anyway — Matt Welsh on the End of Programming

A former professor of computer science and Google engineering lead thinks generative AI will lead to the end of programming within 3 years.

Feb 21st, 2023 10:04am by Richard MacManus

开源Qwen凌晨暴击闭源Claude！刷新AI编程SOTA，支持1M上下文

关注前沿科技 量子位 2025年07月23日 08:24 北京

阿里云开源AI编程模型，新手程序员一天可完成“老师傅”一周工作

陈杨园 第一财经 2025年07月23日 17:29 上海

25位IT大佬亲述：AI「吃掉」程序员！码农黄金时代终结

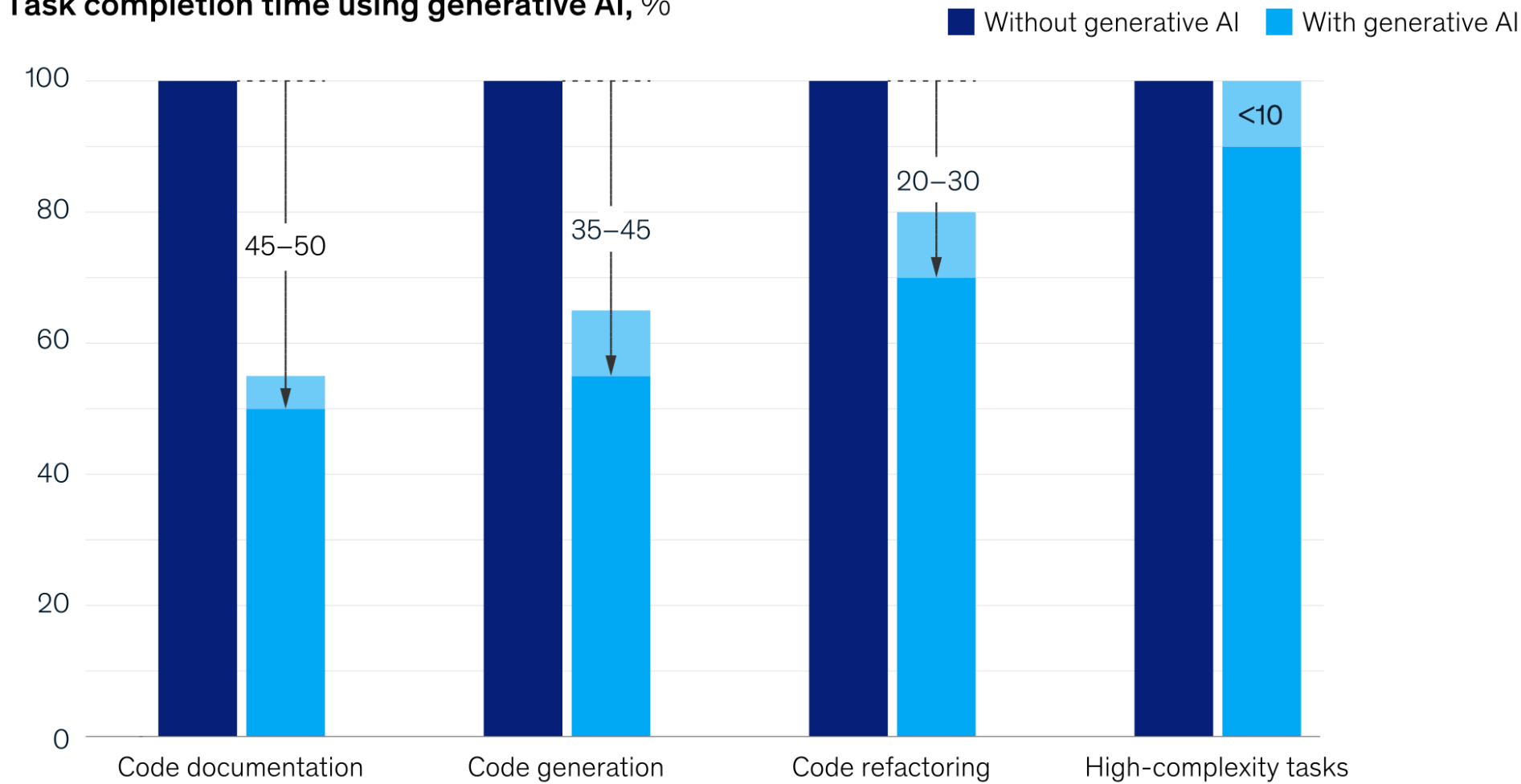
新智元 2025年05月03日 14:59 北京



AI还需沉淀.....

Generative AI can increase developer speed, but less so for complex tasks.

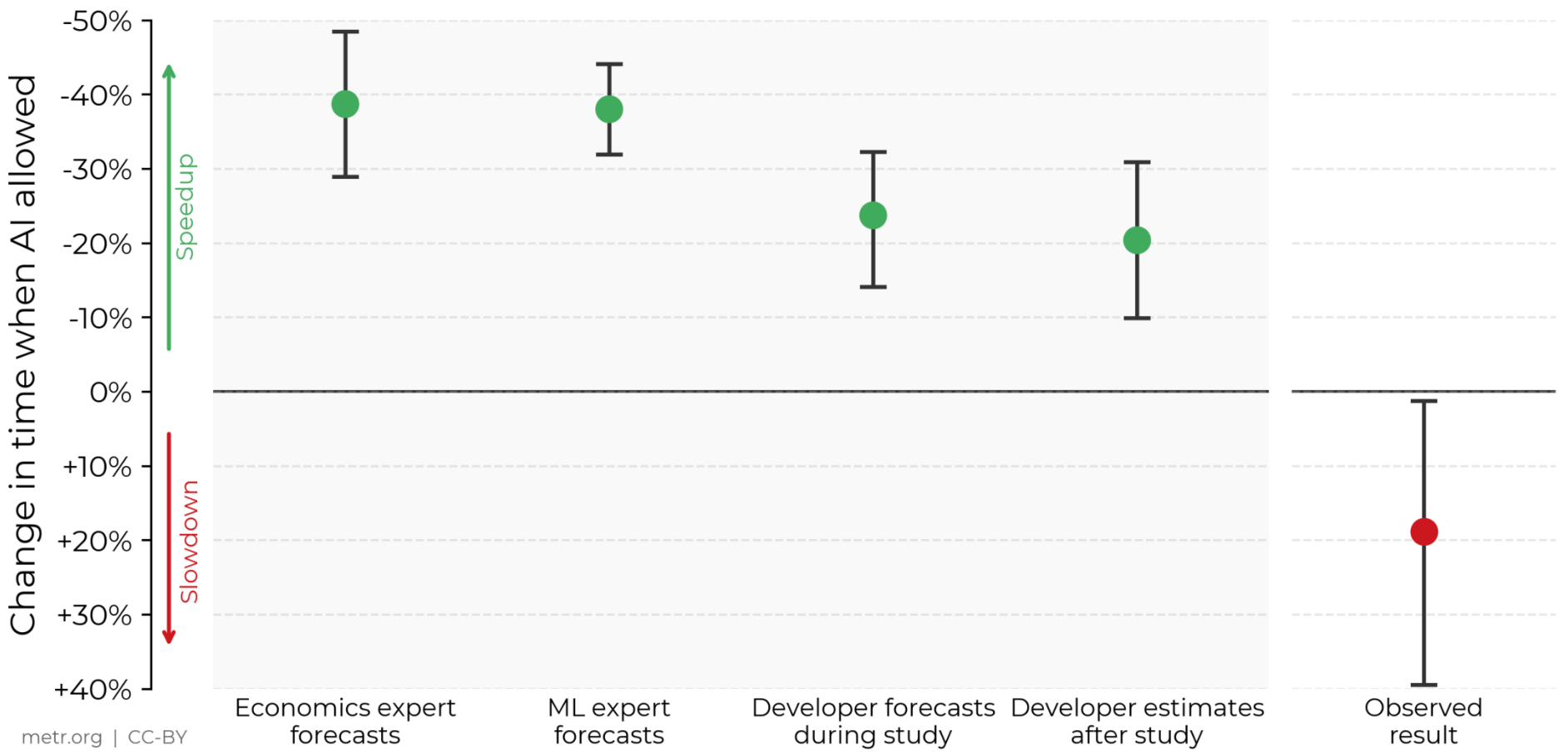
Task completion time using generative AI, %



Against Expert Forecasts and Developer Self-Reports, Early-2025 AI Slows Down Experienced Open-Source Developers



In this RCT, 16 developers with moderate AI experience complete 246 tasks in large and complex projects on which they have an average of 5 years of prior experience.



AI还需沉淀.....

Growing Evidence that AI-Generated Code Optimizes for the Short-Term

越来越多的证据表明 AI 生成的代码优化的是短期效果

AI-assisted code has contributed to a measurable decline in code quality. In 2024, there was a marked increase in the frequency of copy-pasted code, surpassing the number of refactored (or “moved”) lines for the first time. This shift suggests that developers are focusing more on rapid feature implementation rather than long-term maintainability.

AI 辅助代码导致代码质量有所下降。2024 年，复制粘贴代码的频率显著增加，首次超过了重构（或“移动”）的行数。这种转变表明，开发者更加关注快速实现功能，而不是长期的可维护性。

Google’s 2024 DevOps Research and Assessment (DORA) report corroborates these findings, showing a correlation between increased AI adoption and a rising defect rate. Google’s analysis estimated a 7.2% decrease in delivery stability for every 25% increase in AI adoption.

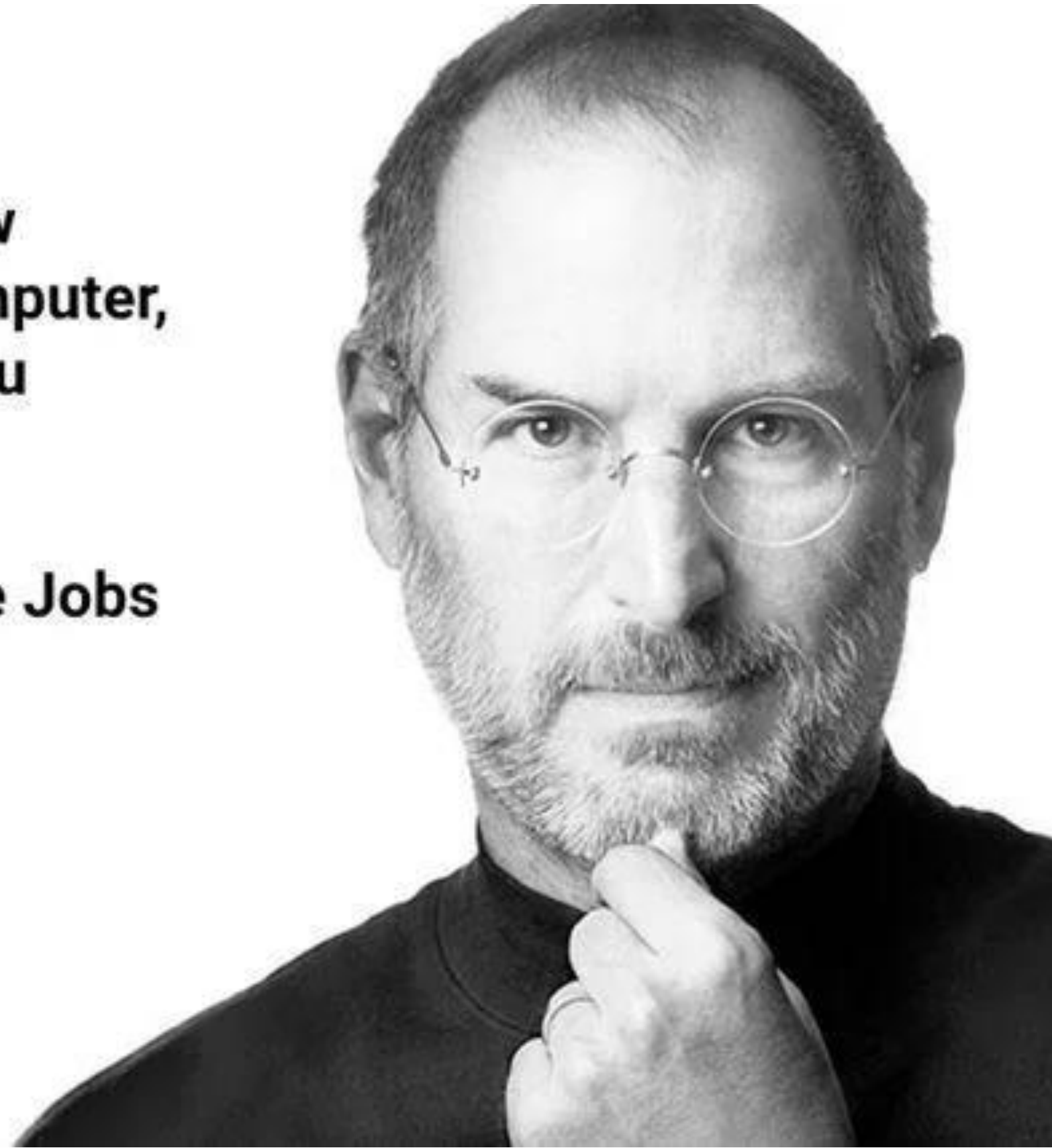
谷歌 2024 年 DevOps 研究和评估（DORA）报告证实了这些发现，显示人工智能采用率的增加与缺陷率的上升之间存在关联。谷歌的分析估计，每增加 25% 的人工智能采用率，交付稳定性将下降 7.2%。

<https://www.jonas.rs/2025/02/09/report-summary-gitclear-ai-code-quality-research-2025.html>

为什么我们要学编程？

**"Everyone should know
how to program a computer,
because it teaches you
how to think."**

Steve Jobs



计算机如何解决问题

□ “请给我一杯水”

1. 转身走进厨房
2. 找到一个水壶
3. 找到一个杯子
4. 把水倒入杯子
5. 拿着杯子回来

计算机只能执行清晰具体的指令！

计算机的组成

□ 计算机主要由CPU、内存、硬盘、输入/输出装置组成

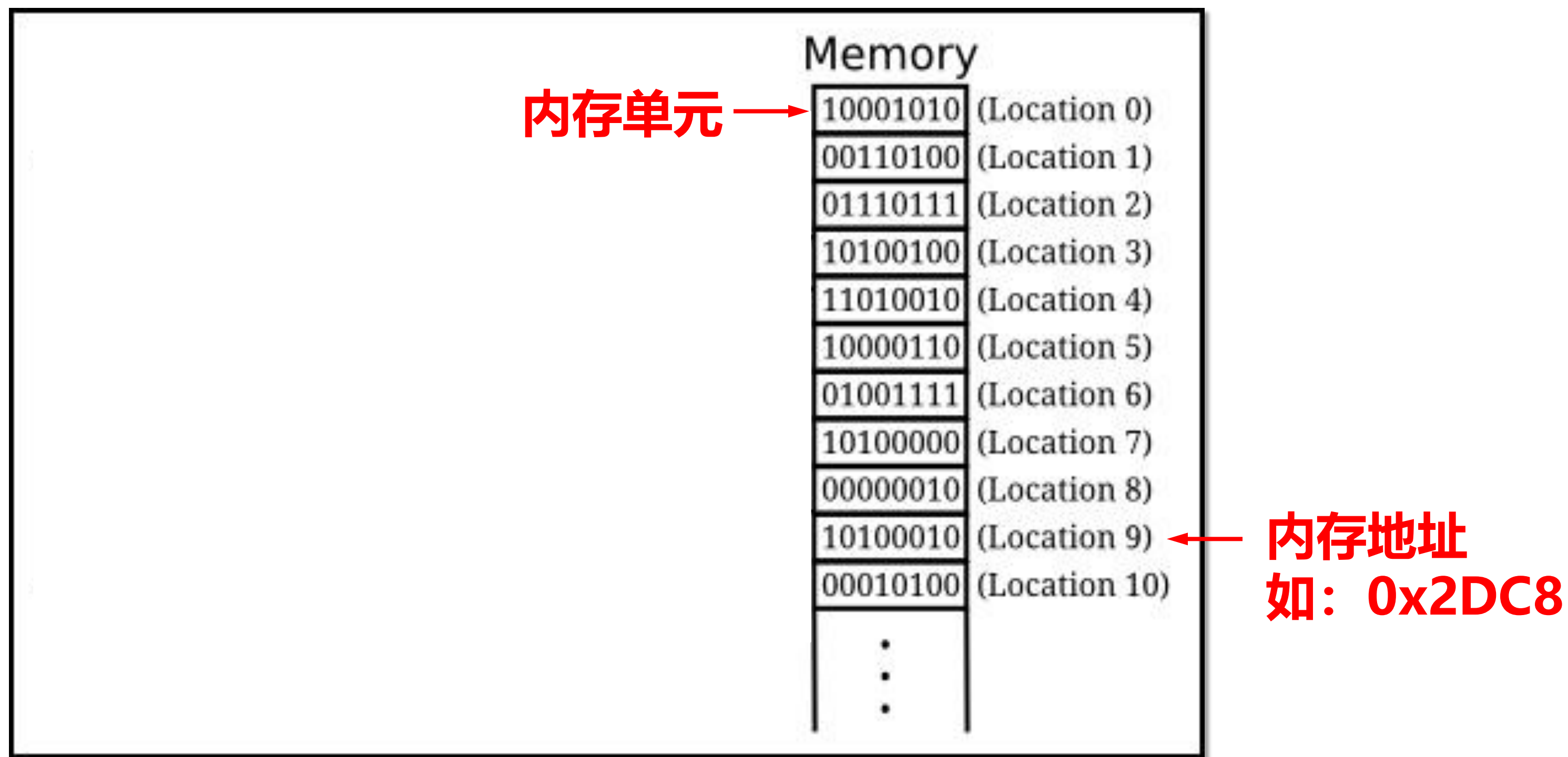


CPU的工作原理

- 从内存中获取并执行一条指令，然后再从内存中获取并执行下一条指令

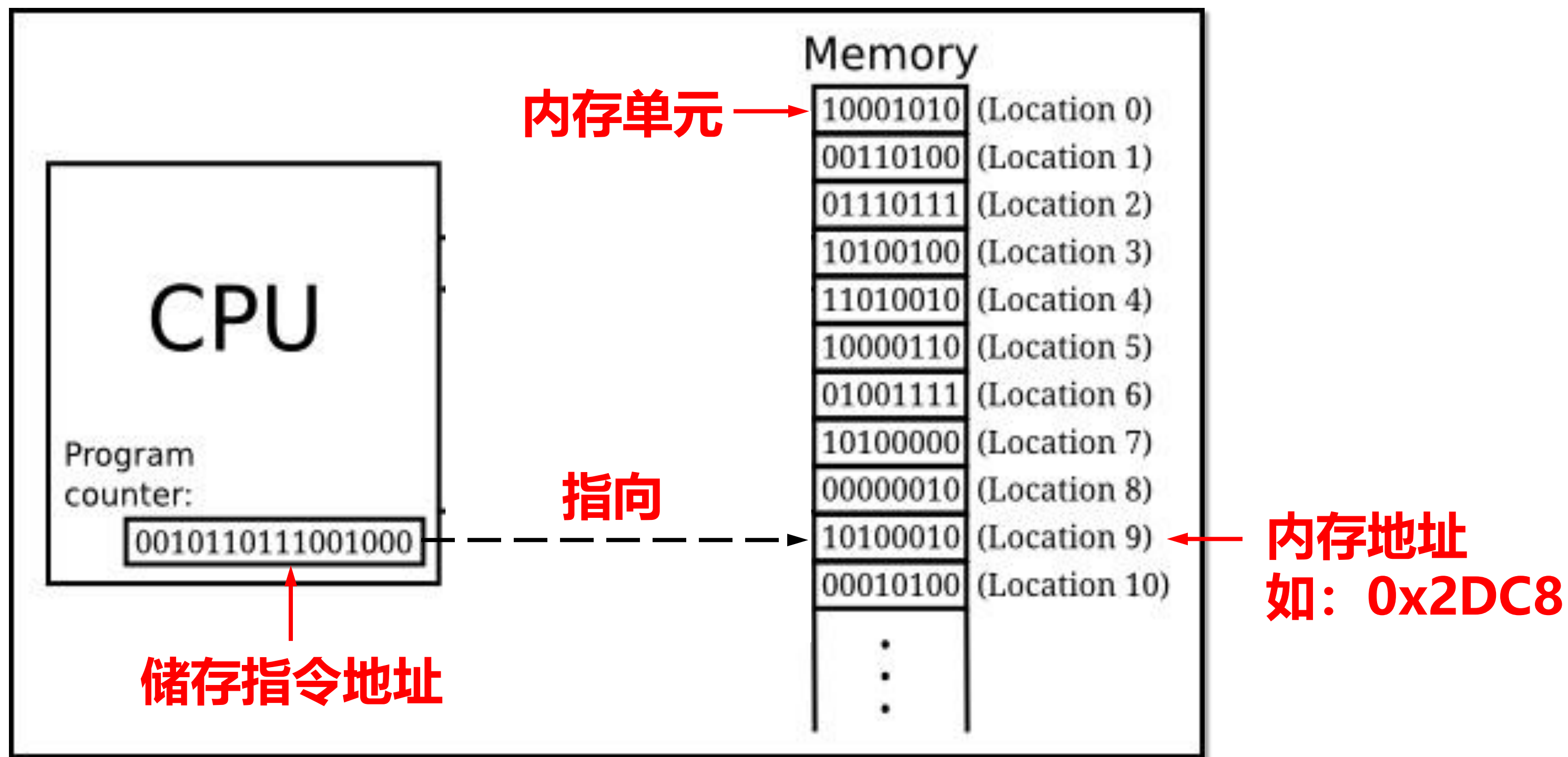
CPU的工作原理

- 内存由一个个大小固定（1字节）的**内存单元**组成，每个单元有唯一地址



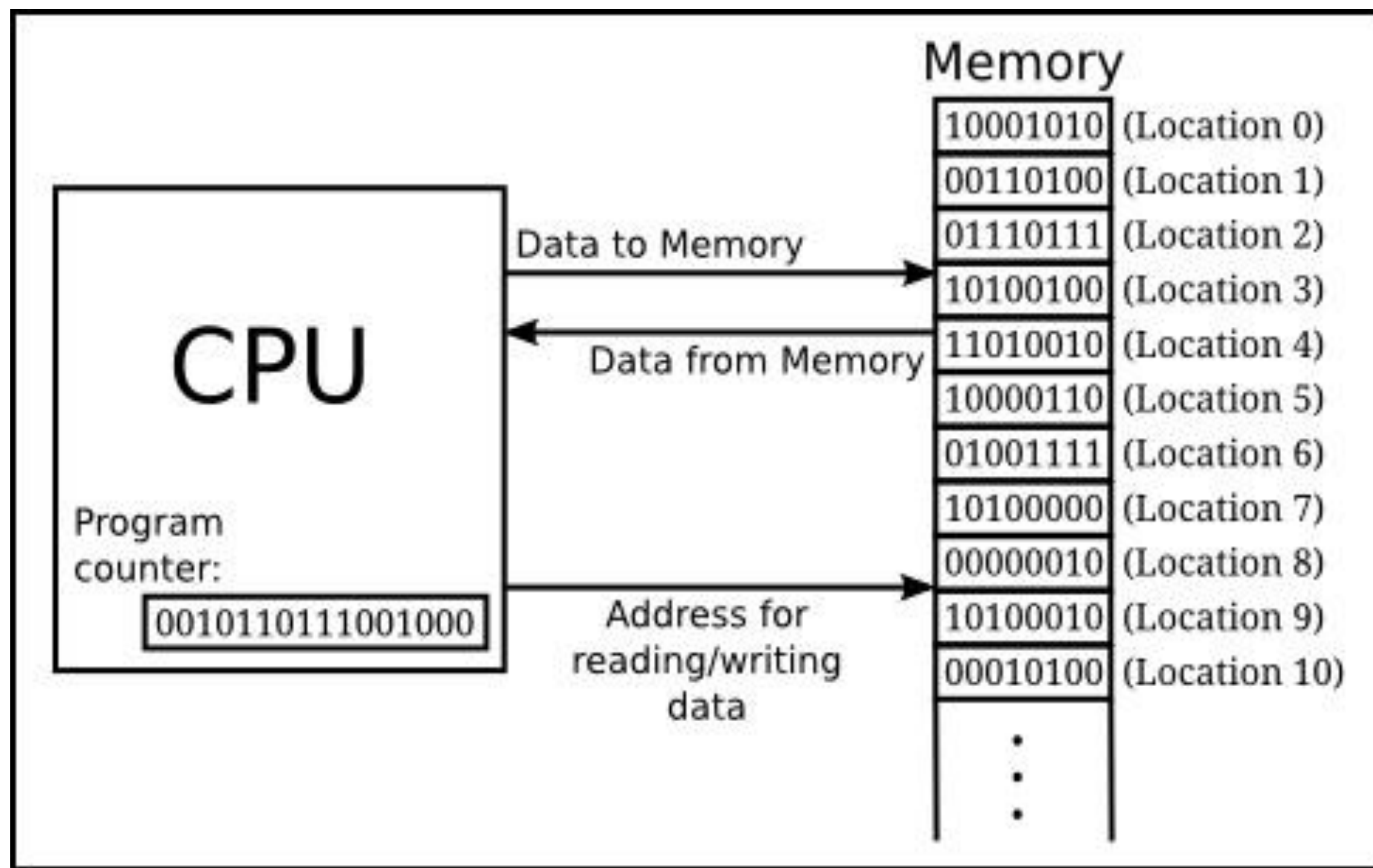
CPU的工作原理

- ❑ 程序计数器 (Program Counter) 存储当前要执行指令所在的内存地址



CPU的工作原理

□ CPU指令周期：取指→译码→执行→存储



如何让计算机解决问题？

□ 需为计算机提供其能理解的**特定的指令列表——程序**

□ 两数相加

1. 从**内存位置2000**把一个数字拷贝到**寄存器1**
2. 从**内存位置2004**把另一数字拷贝到**寄存器2**
3. 把**寄存器2**中内容与**寄存器1**中的内容相加，
结果存储在**寄存器1**中
4. 把**寄存器1**中的内容拷贝到**内存位置2008**

编程语言：计算机能听懂的语言

□ 编程语言是用来**编写计算机程序的标准化语言**

机器语言：最“直接的”编程语言

❑ 在计算机的内部，主要的电子元件都是由晶体管构成的，这就意味着计算机只能理解0（低电平）和1（高电平）

❑ 两数相加

1. 从内存位置2000把一个数字拷贝到寄存器1
2. 从内存位置2004把另一数字拷贝到寄存器2
3. 把寄存器2中内容与寄存器1中的内容相加，结果存储在寄存器1中
4. 把寄存器1中的内容拷贝到内存位置2008



```
01 01 07 D0
01 02 07 D4
03 01 02
02 01 07 D8
```

机器语言实现

机器语言：最“直接的”编程语言

❑ 计算32和26最大公约数：

1. 把 u 设为 32

2. 把 v 设为 26

3. 当 v 不是 0 的时候，重复下面的步骤：

1. 计算 u 除以 v 的余数，记作 temp

2. 把 u 的值改成 v

3. 把 v 的值改成 temp

1. 循环结束后，u 就是最大公约数

2. 输出 u

CF	FA	ED	FE	07	00	00	01	03	00	00	00	01	00	00	00	03	00	00	00	F0	01	00	00	00	20	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	--

机器语言有什么问题？

- ❑ 机器语言**繁冗又难理解**，难到程序员只能靠机器语言来和计算机打交道吗？
- ❑ 想象一下，假设你完全不会俄语，但你又需要与俄国人打交道，有什么办法呢？
- ❑ 答案是**借助一种中间语言**，比如英语，实现汉语→英语→俄语的交流



汇编语言：对机器语言的“翻译”

□ 借助汇编语言，我们可以将一些常见的机器指令转化成人類可以理解的**助记符**，比如规定：

■ add = 000000

■ sub = 100010

■ mul = 000010

■ div = 011010

31	26	25	21	20	16	15	11	10	6	5	0	
SPECIAL 000000	rs		rt		00000		00000		DIV 011010			div指令
SPECIAL 000000	rs		rt		00000		00000		DIVU 011011			divu指令



汇编语言实现 “两数相加”

□ 两数相加

1. 从内存位置2000把一个数字拷贝到寄存器1
2. 从内存位置2004把另一数字拷贝到寄存器2
3. 把寄存器2中内容与寄存器1中的内容相加，结果存储在寄存器1中
4. 把寄存器1中的内容拷贝到内存位置2008

01 01 07 D0
01 02 07 D4
03 01 02
02 01 07 D8

机器语言实现

```
mov eax, dword [0x07D0]
mov ebx, dword [0x07D4]
add eax, ebx
mov dword [0x07D8], eax
```

汇编语言实现 (x86)

汇编语言有什么问题？

- 汇编语言与底层的处理器架构绑定，换机器就需要重写，使得程序移植性差
- 能否可以用更“抽象”的方式表达我们的想法？
 - 如“两数相加”能否直接表示为： $\text{sum} = a + b;$



本门课的主角：C语言

- C语言则是对汇编语言的进一步抽象，不受限于特定的CPU和指令集，使得人类更容易理解

```
01 01 07 D0
01 02 07 D4
03 01 02
02 01 07 D8
```

机器语言实现

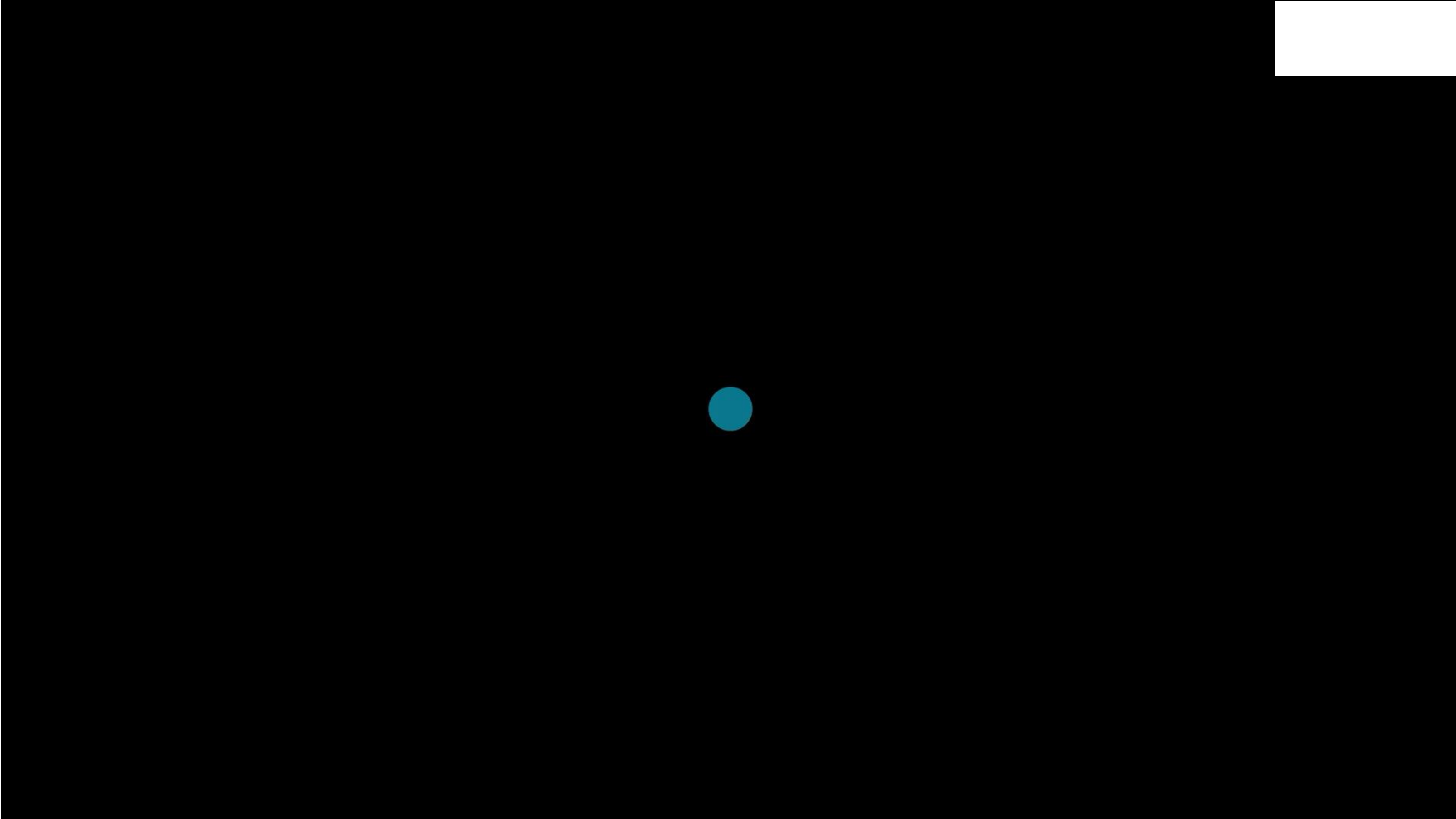
```
mov eax, dword [0x07D0]
mov ebx, dword [0x07D4]
add eax, ebx
mov dword [0x07D8], eax
```

汇编语言实现 (x86)

```
sum = a + b;
```

C语言实现

C语言简史



https://www.bilibili.com/video/BV1Eb411b716/?spm_id_from=333.337.search-card.all.click&vd_source=ba8bb8c9887aff7403e9af7acc7e5ad9

C语言简史

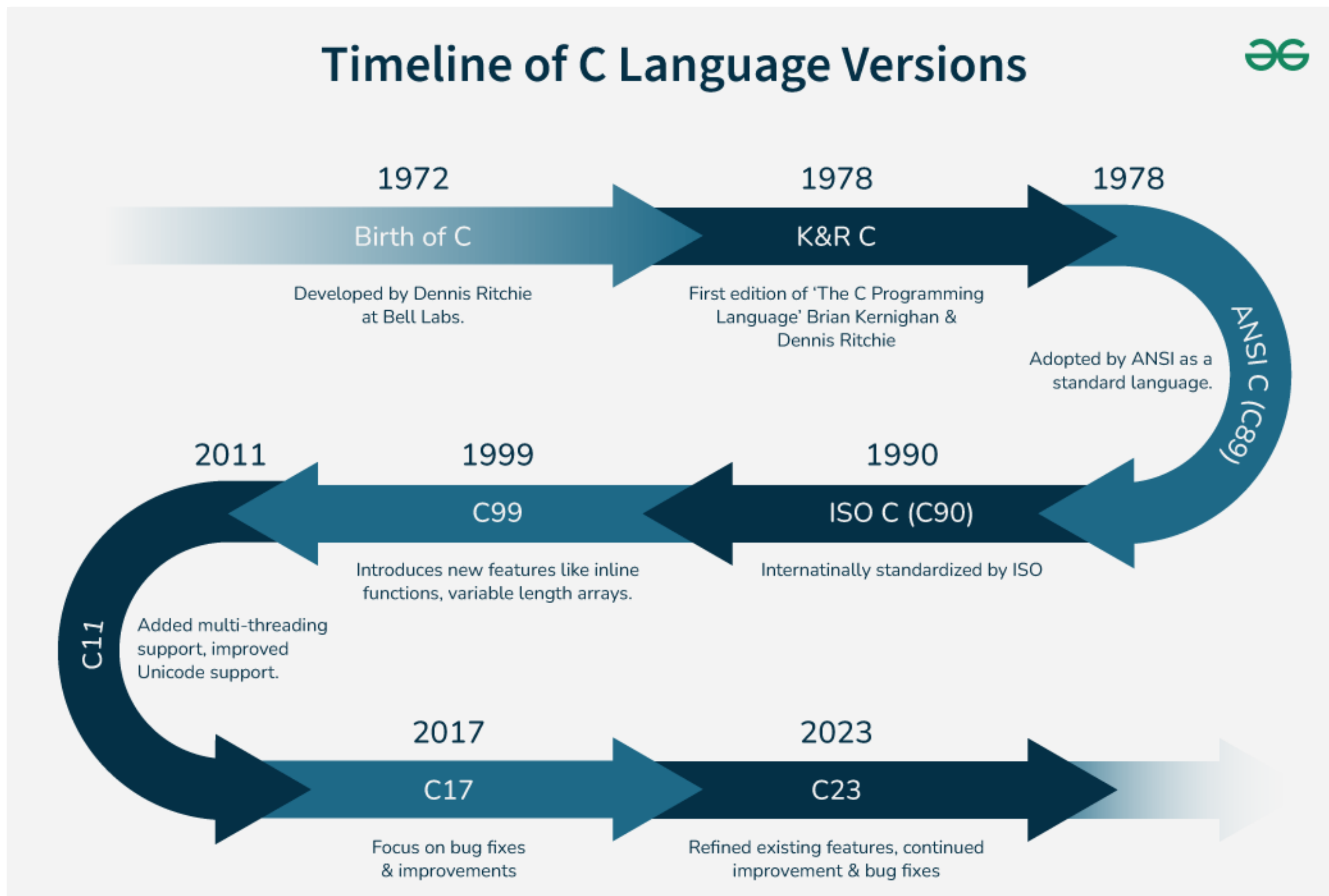


肯·汤普森 (Ken Thompson) 和丹尼斯·里奇 (Dennis Ritchie)

C语言标准

- ❑ 最初的C语言只能在大型计算机上执行，被移植到微机上来出现了许多不同的C语言版本
- ❑ 为了改变这种情况，1983年美国国家标准化协会（ANSI）为C语言制定了第一个正式C标准（ANSI C），定义了C标准和C标准库
- ❑ 1994年，ANSI/ISO联合委员会（C9X委员会）开始修订C标准，最终在1999年发布了C99标准
- ❑ 所有的当代编译器都支持C99，本门课主要遵循C99标准

C语言标准

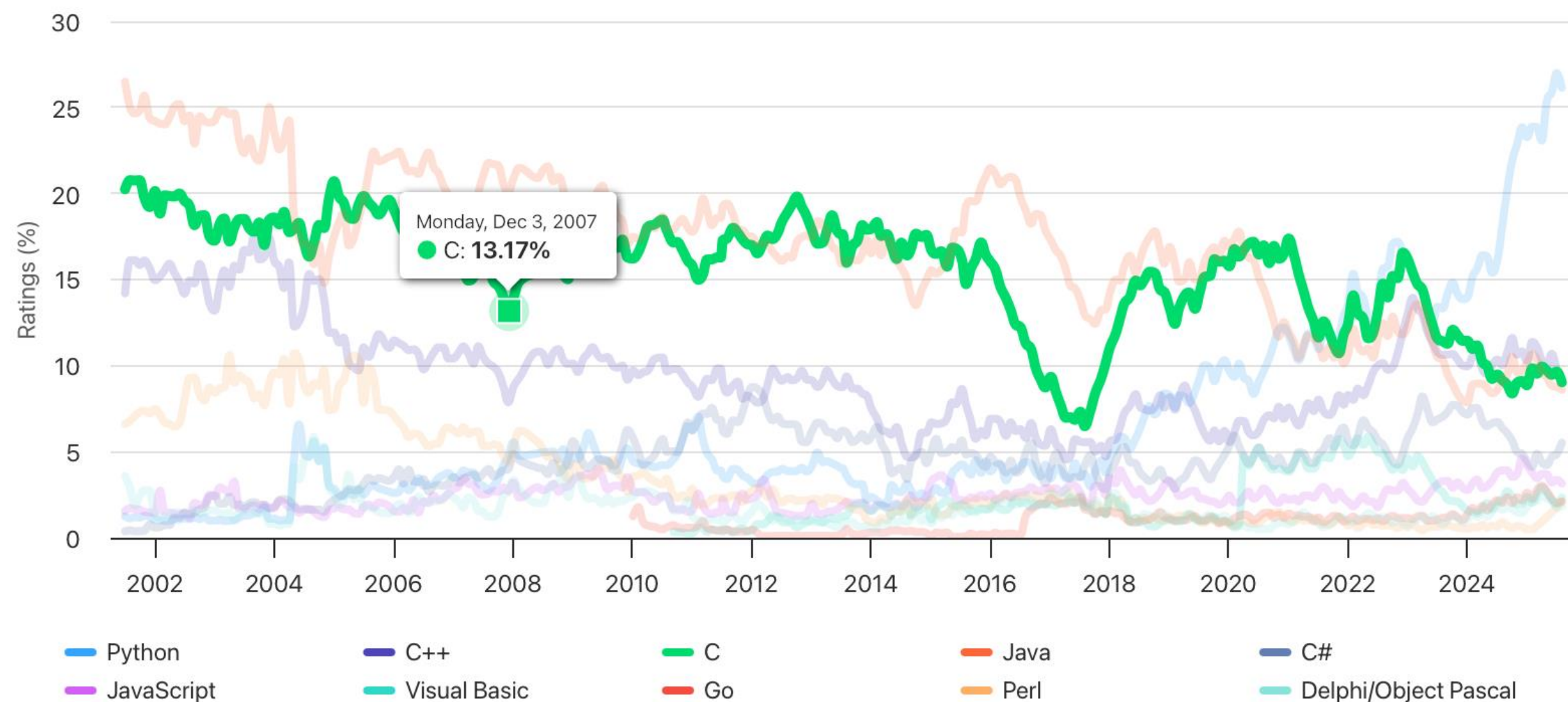


为什么学C?

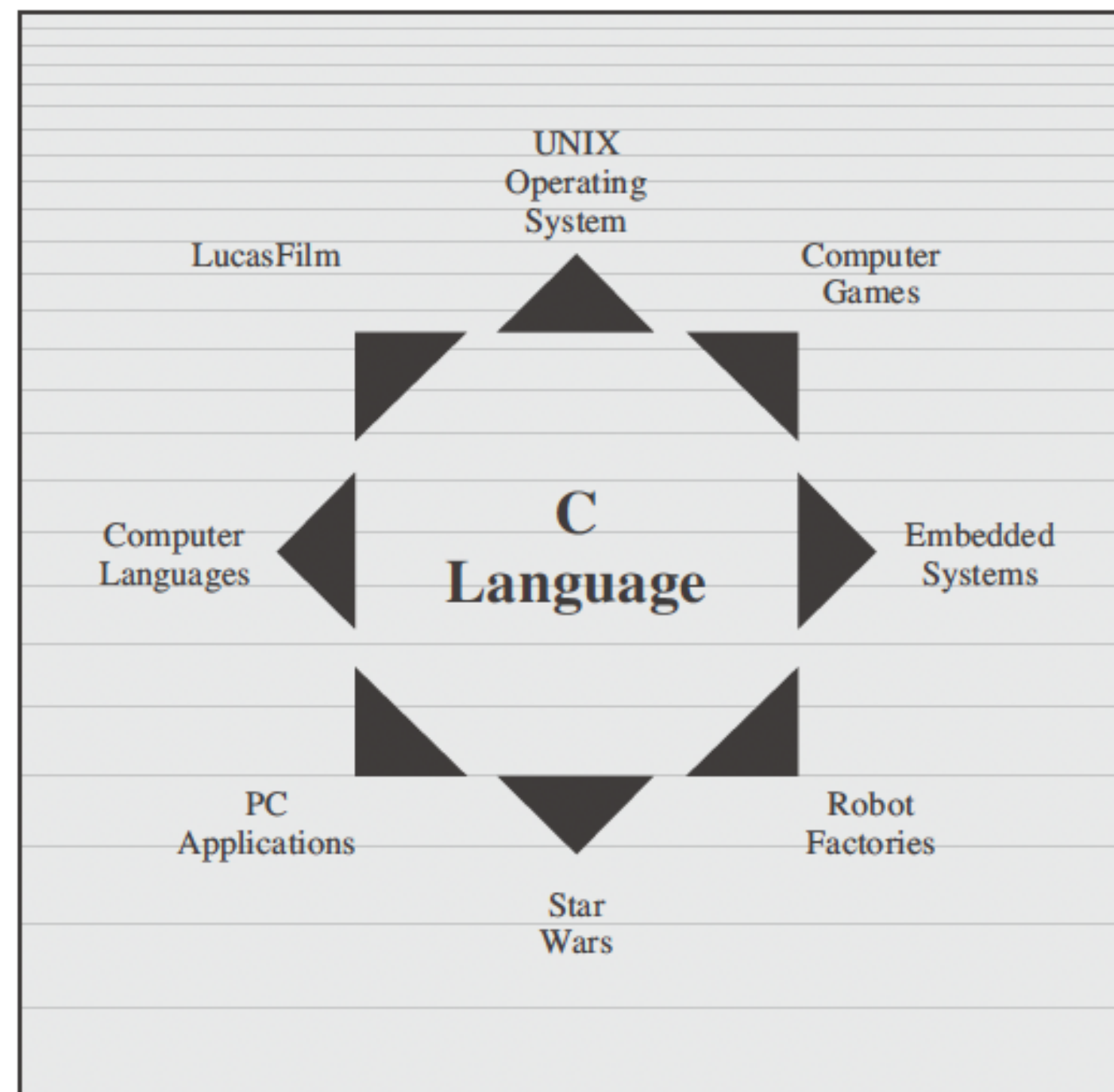
□ 过去40年最重要、最流行的编程语言之一

TIOBE Programming Community Index

Source: www.tiobe.com



<https://www.tiobe.com/tiobe-index/>



为什么学C?

- 过去40年最重要、最流行的编程语言之一
- C的优势
 - **高效**：具备CPU、内存微调控制能力
 - **用户友好**：易读，易写
 - **灵活**：不同操作系统间可移植性强
- 现代编程语言语法差异很小
 - 几乎都是C-like语言

```
#include <stdio.h>

int main()
{
    printf("Hello World!\n");

    return 0;
}
```

```
class Hello {
    public static void main(String[] arg) {
        System.out.println("Hello World!");
    }
}
```

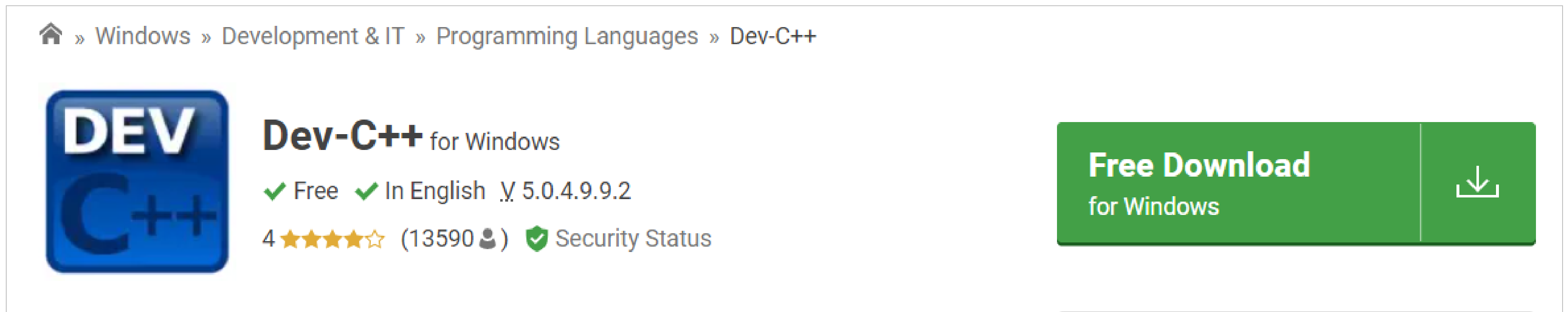
```
print "Hello World!\n"
```

C语言编程环境设置：IDE

- ❑ 集成开发环境（IDE, Integrated Development Environment）是用于提供程序开发环境的应用程序，一般包括代码编辑器、编译器、调试器和图形用户界面等工具
- ❑ 对于刚刚学习C语言的新手，推荐使用DEV-C++
- ❑ 可以在本课程微信群下载

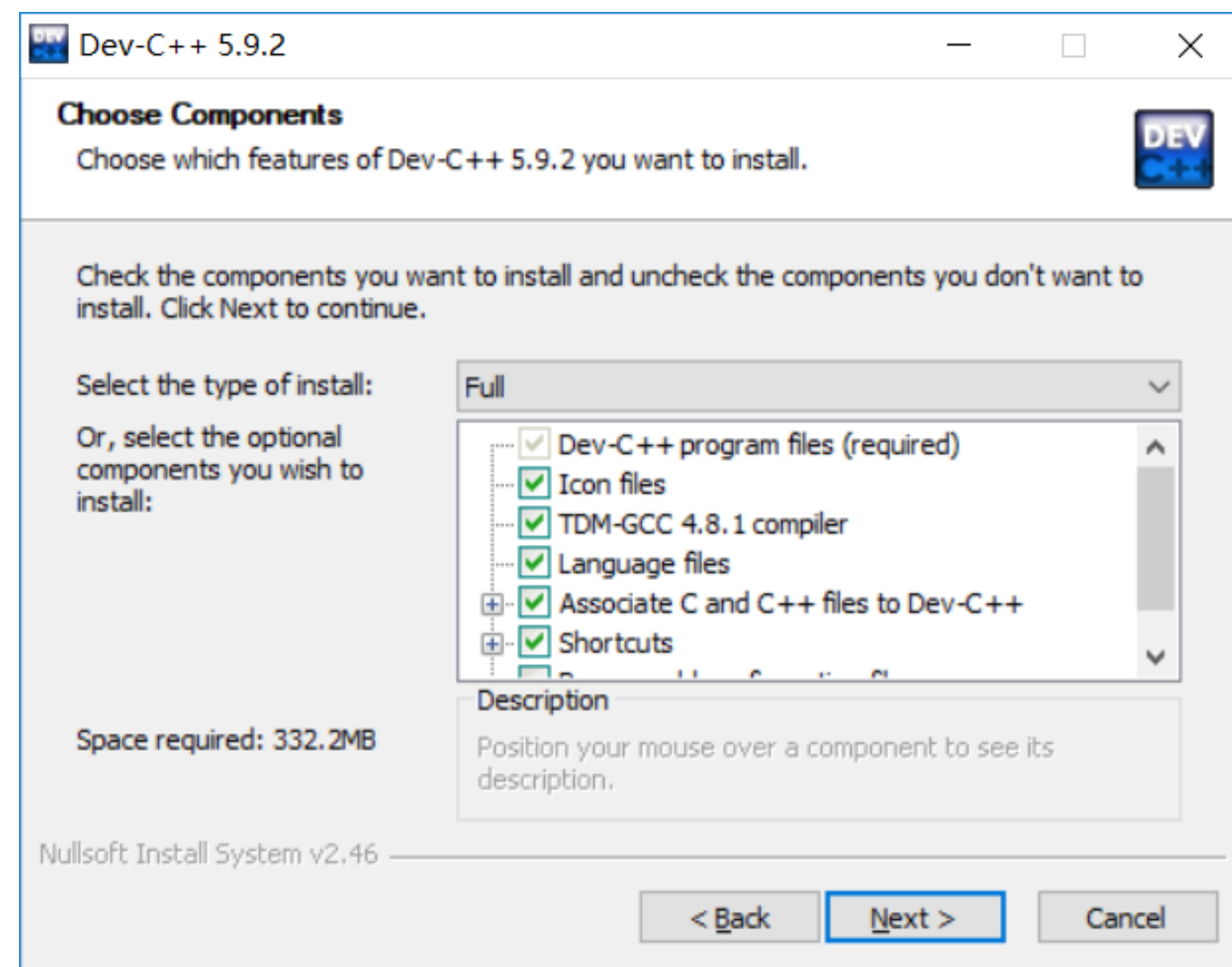
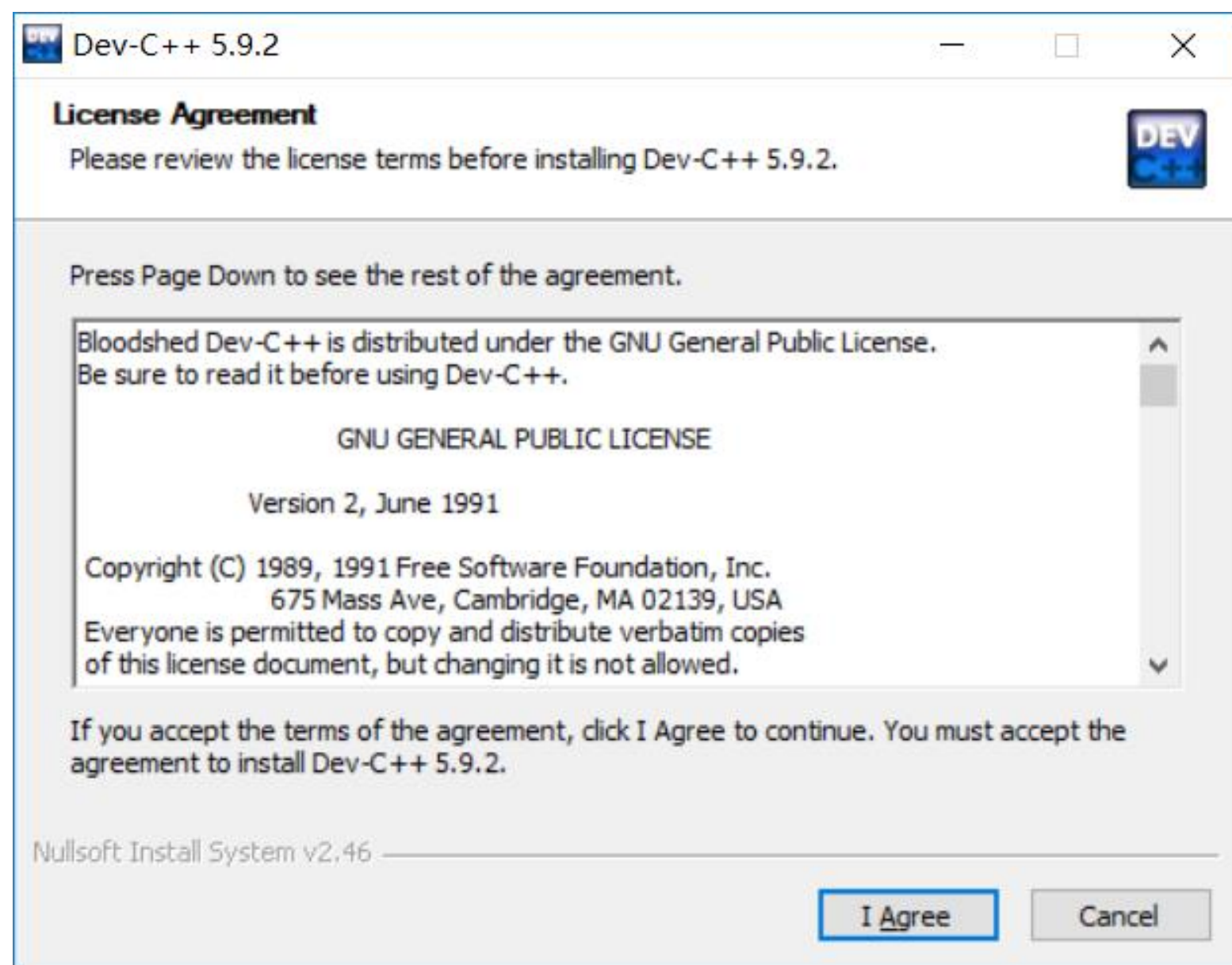
DEV-C++安装示例（一）

- 打开DEV-C++官网，点击download下载安装包



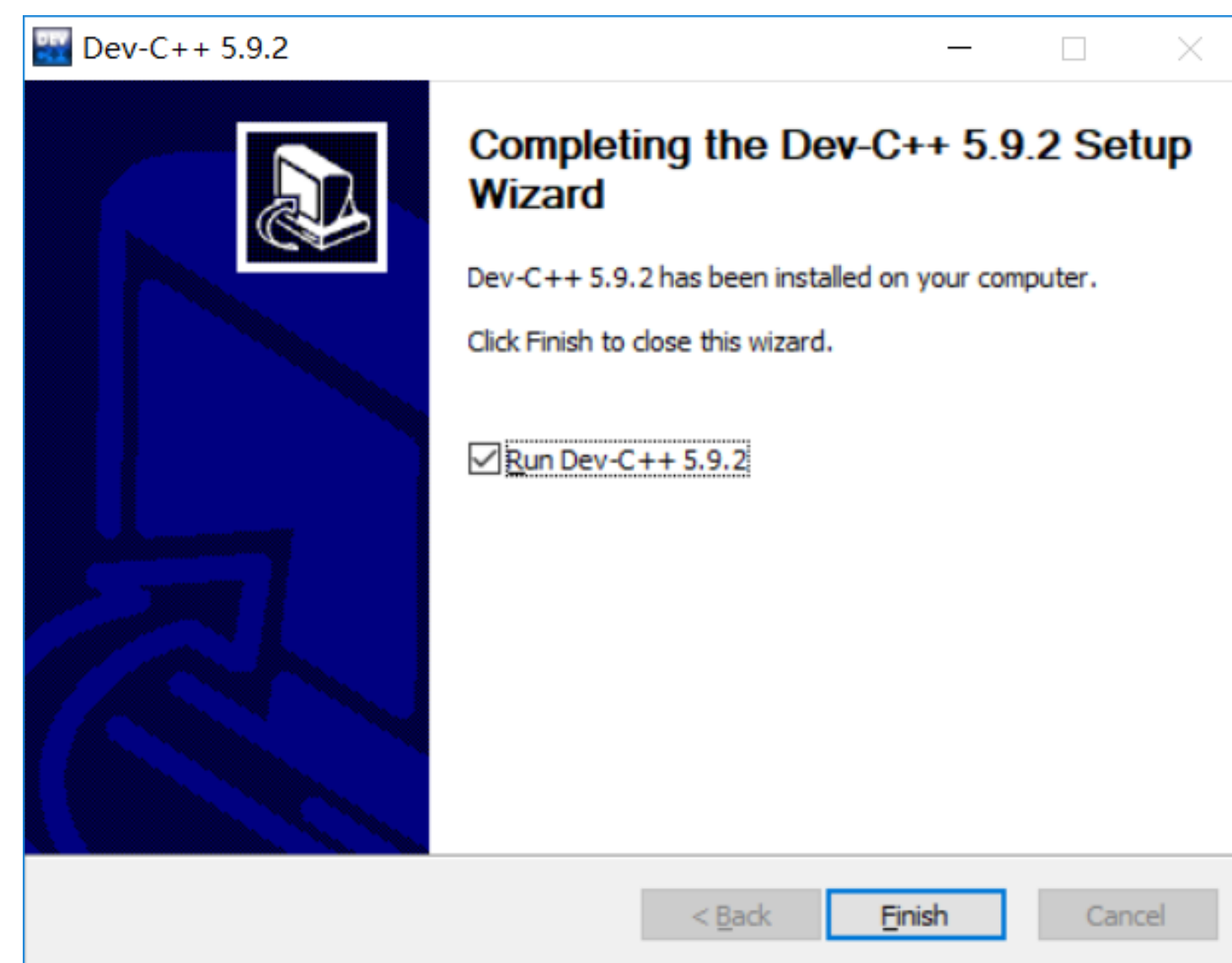
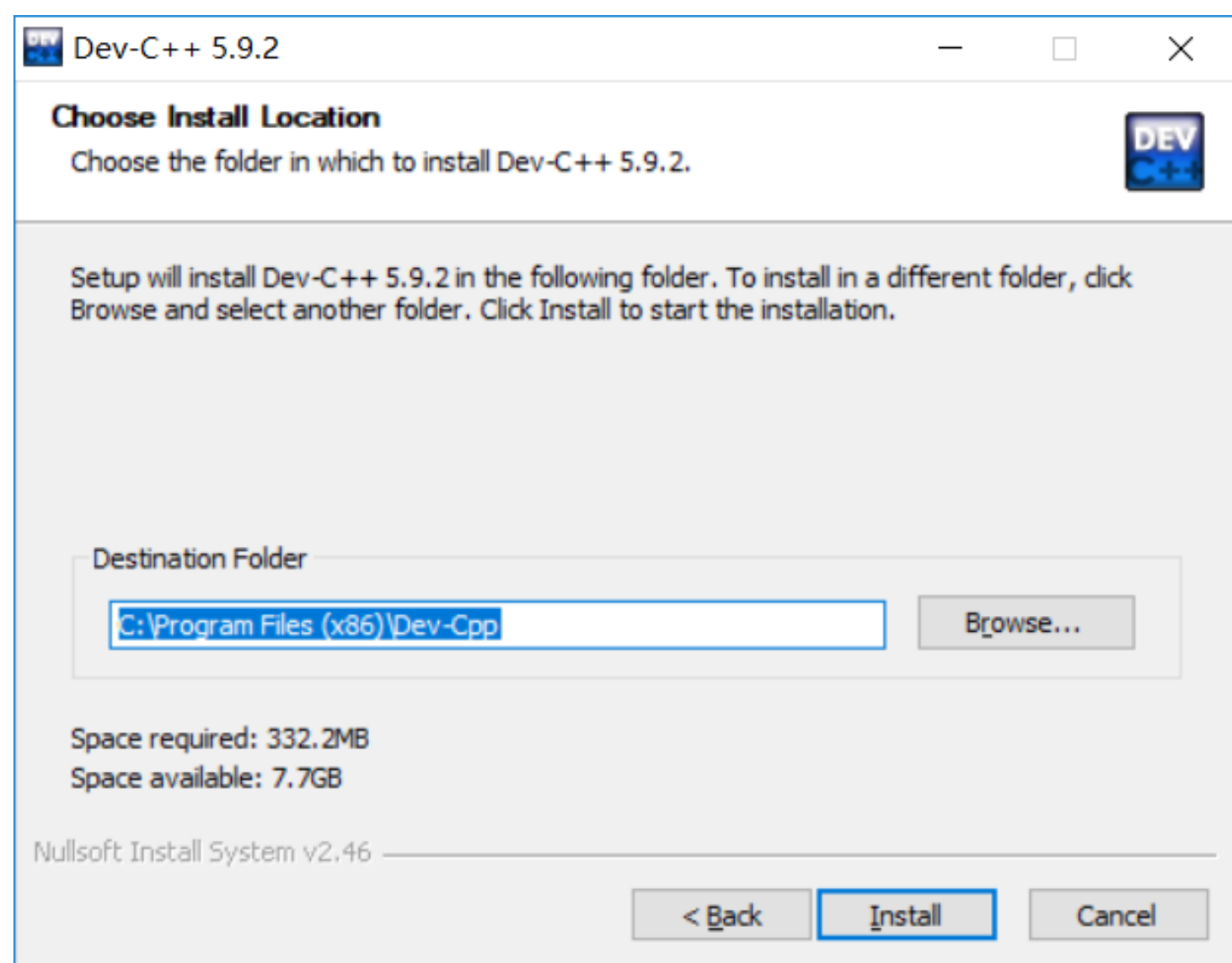
DEV-C++安装示例（二）

- 打开安装包，一路同意即可



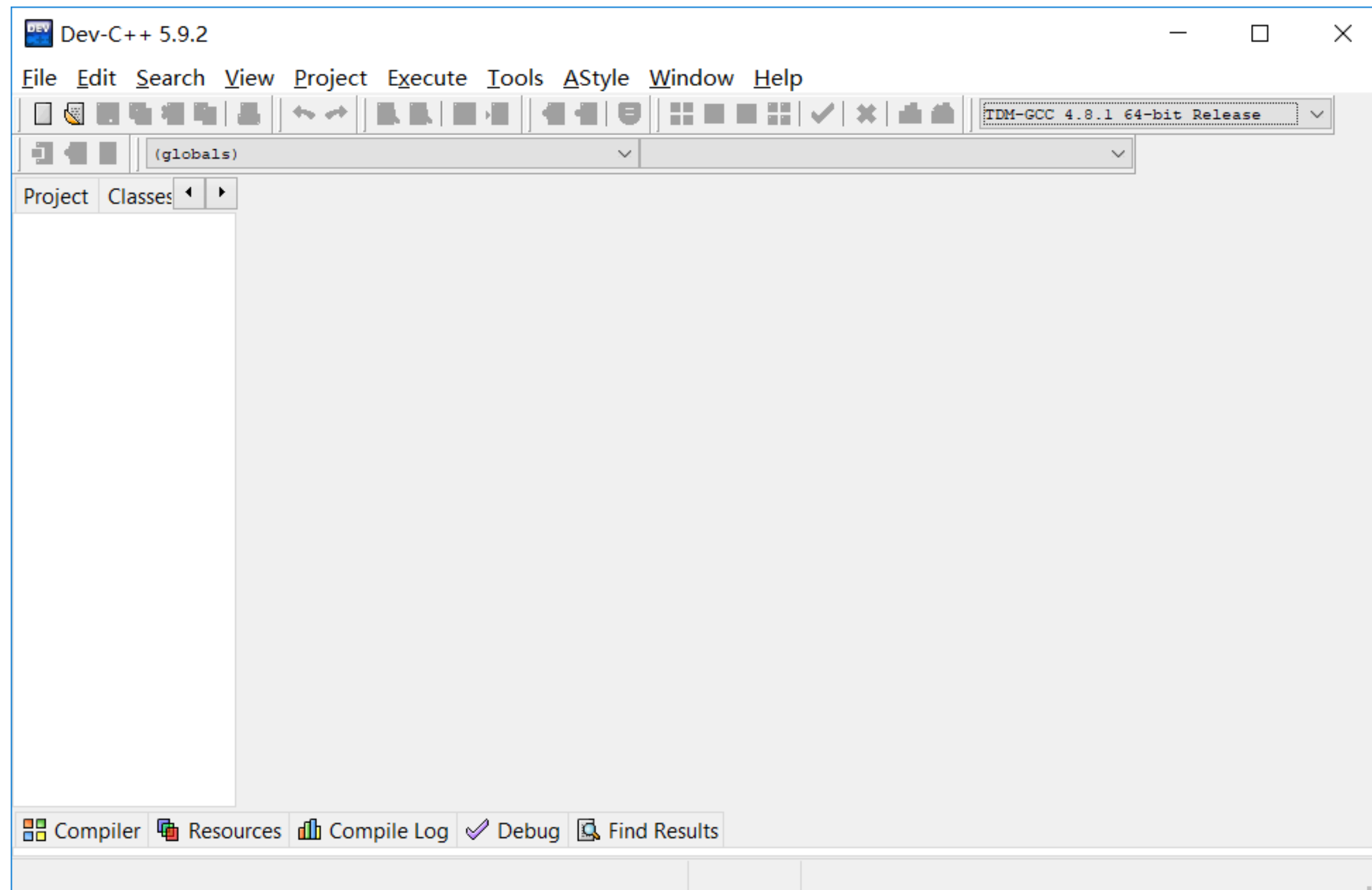
DEV-C++ 安装示例 (三)

□ 一路同意即可



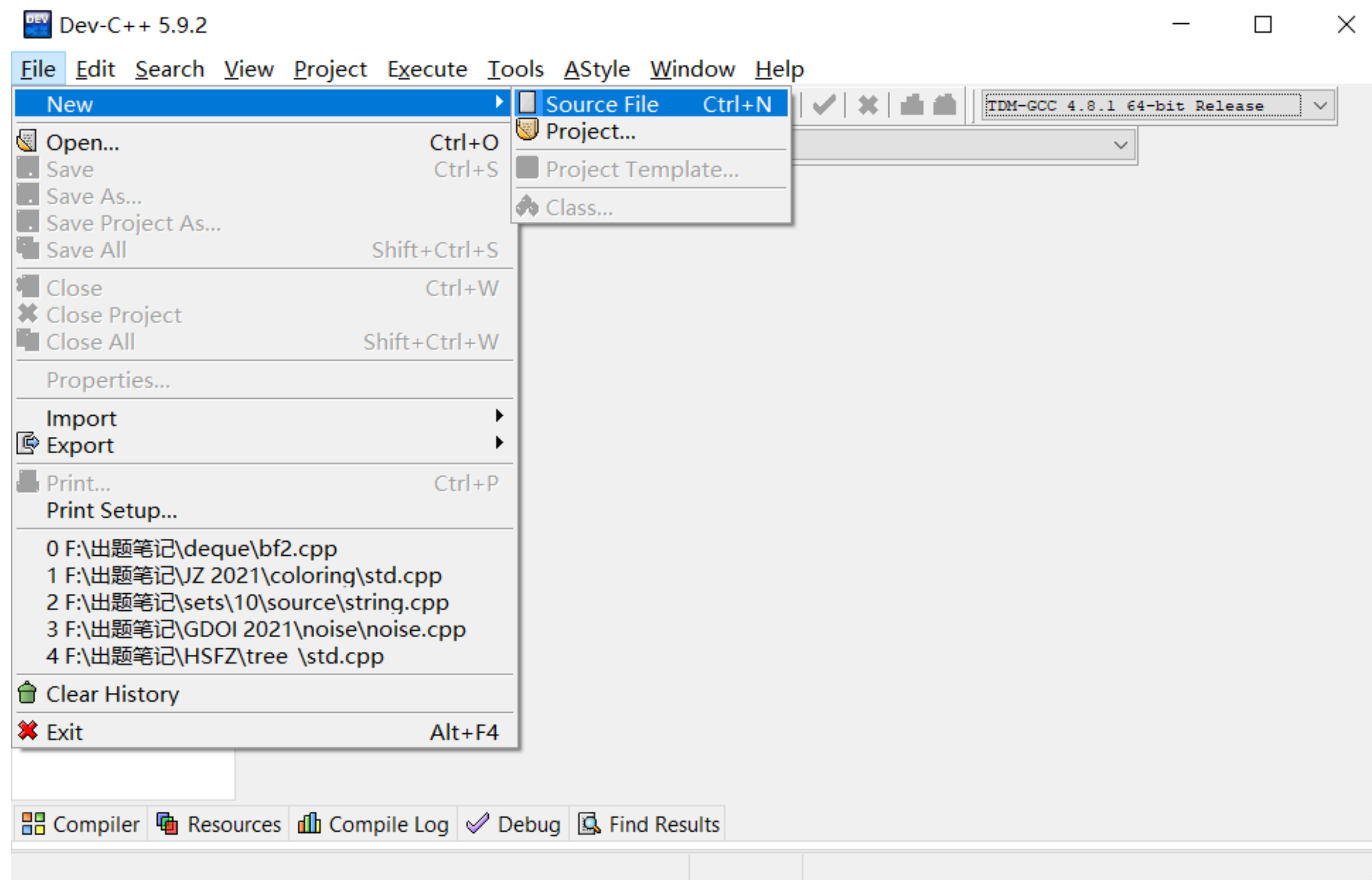
DEV-C++安装示例（四）

- ❑ 如下图即为安装完成，打开界面的样子



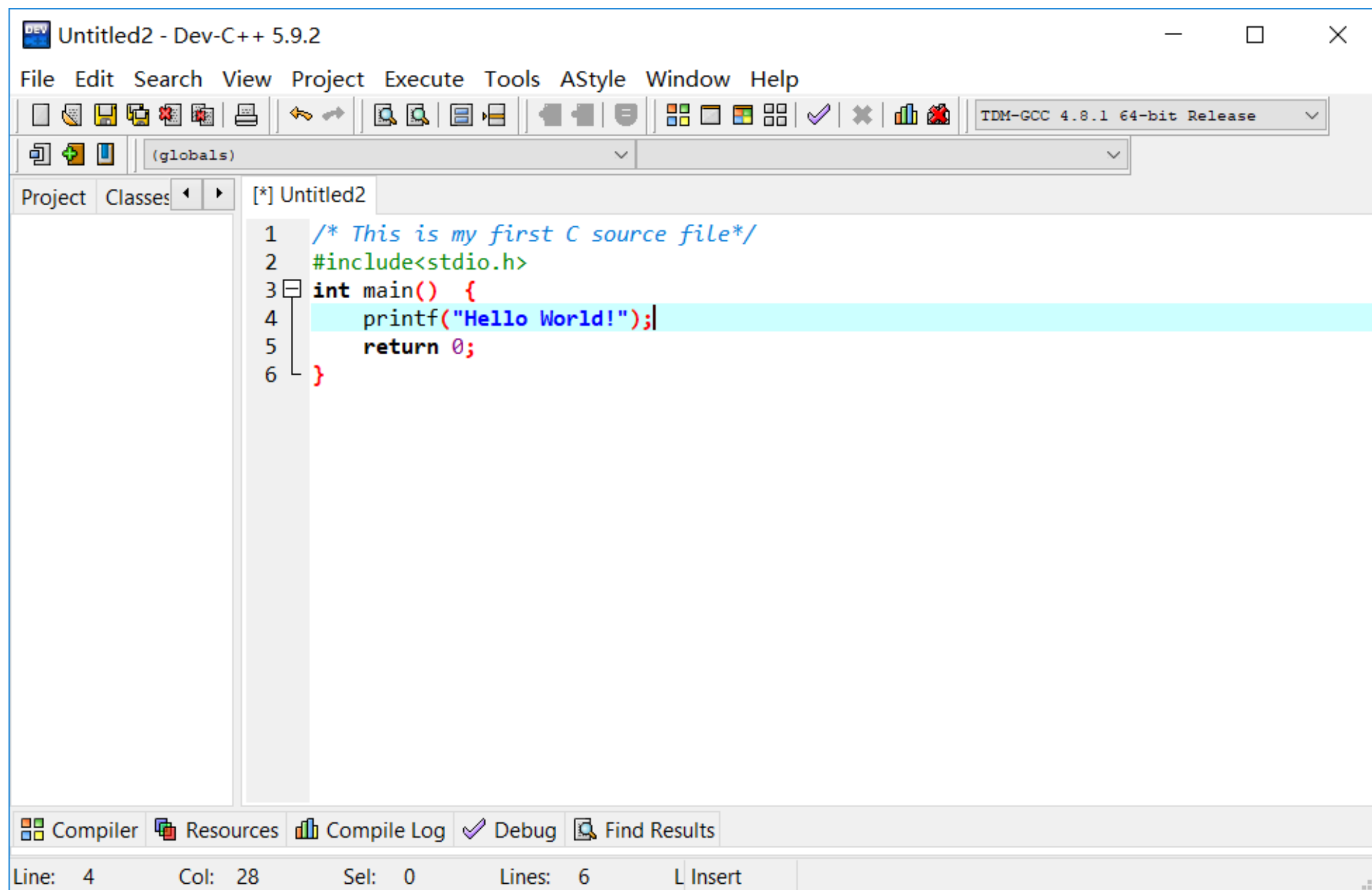
第一个C语言程序（一）

- ❑ 点击左上角的File/New/Source File（Ctrl+N也可）创建新文件



第一个C语言程序（二）

- 在新打开的编辑界面中输入如下代码



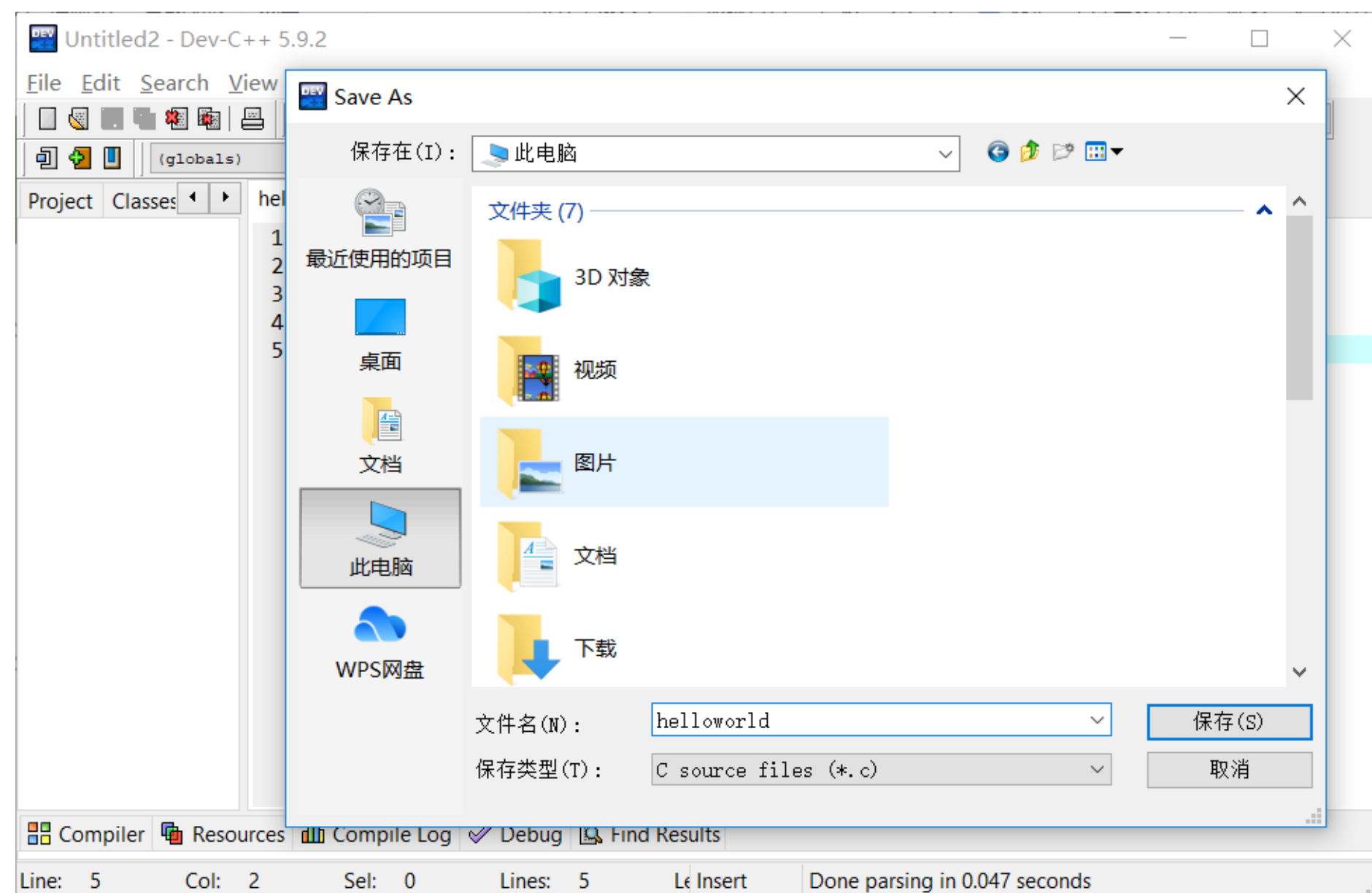
The screenshot shows the Dev-C++ 5.9.2 IDE with a new file named 'Untitled2'. The code editor contains the following C program:

```
1  /* This is my first C source file*/  
2  #include<stdio.h>  
3  int main() {  
4      printf("Hello World!");  
5      return 0;  
6  }
```

The code is color-coded: comments are blue, preprocessor directives are green, keywords are red, and identifiers/strings are black. The line containing the printf statement is highlighted in light blue. The status bar at the bottom indicates the cursor is at Line 4, Column 28, and the file is 6 lines long.

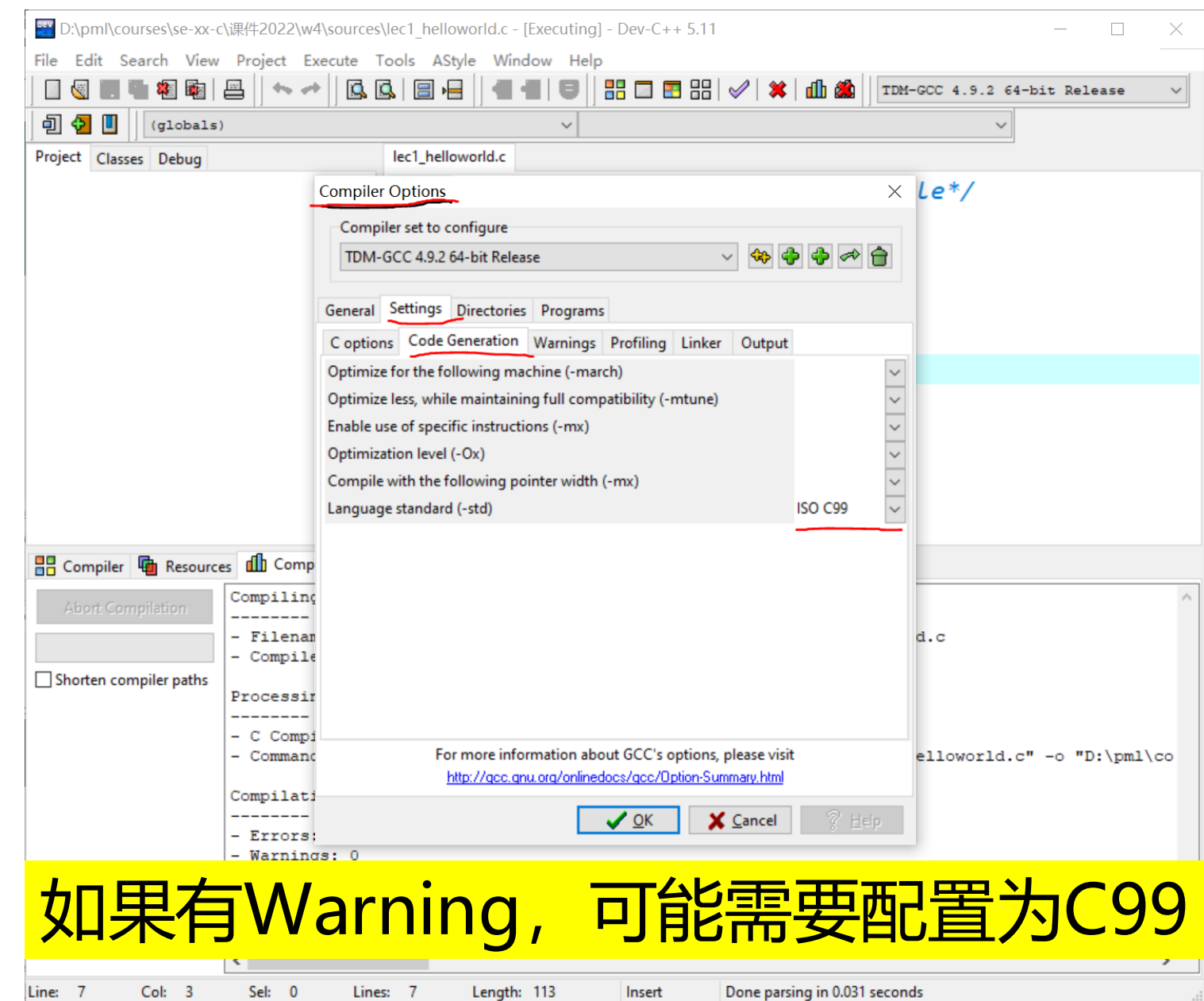
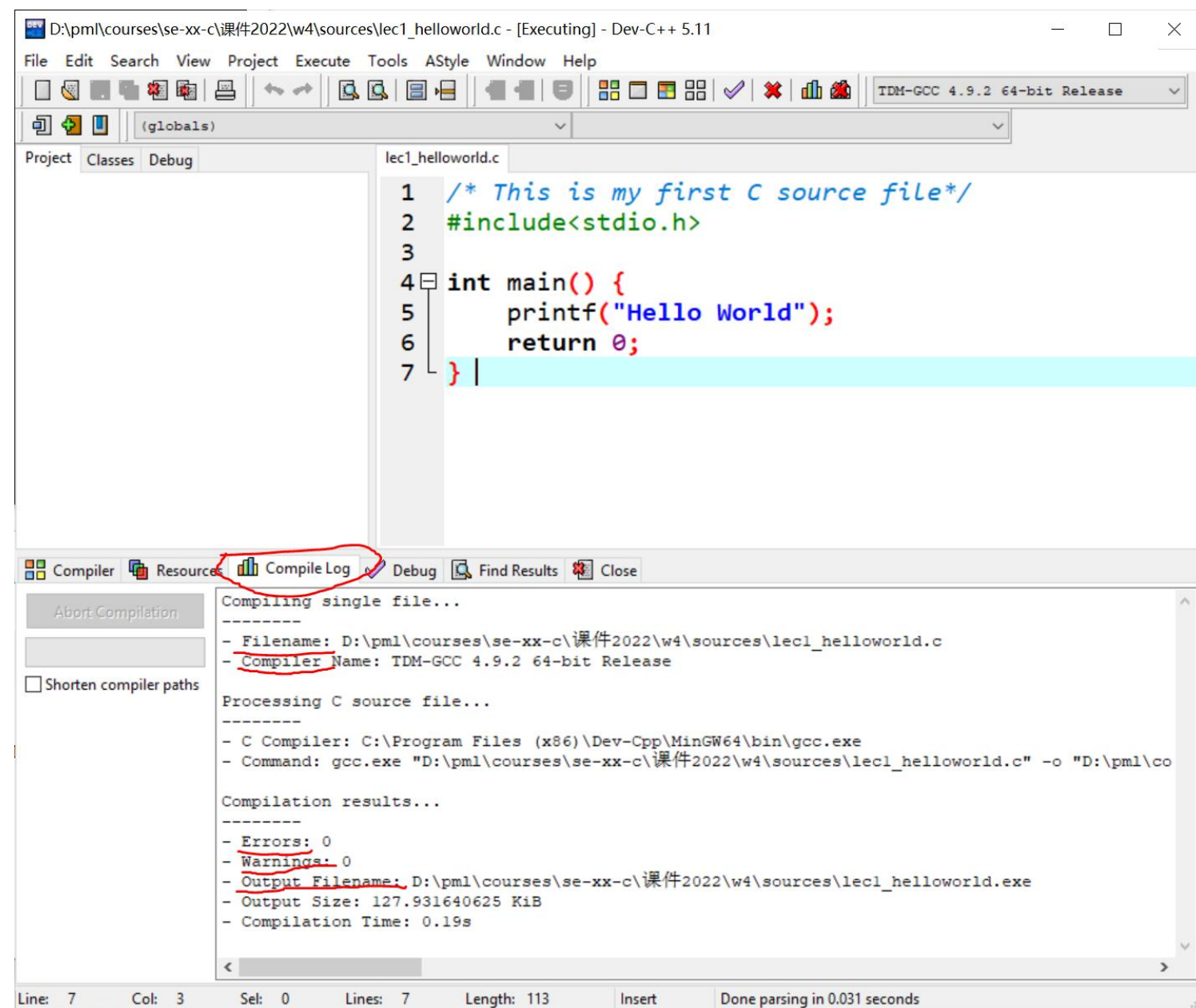
第一个C语言程序（三）

- ❑ 点击左上角的File/Save（Ctrl+S也可）保存文件
- ❑ 文件名填写为 `helloworld.c`，保存类型选择为 “C source file”



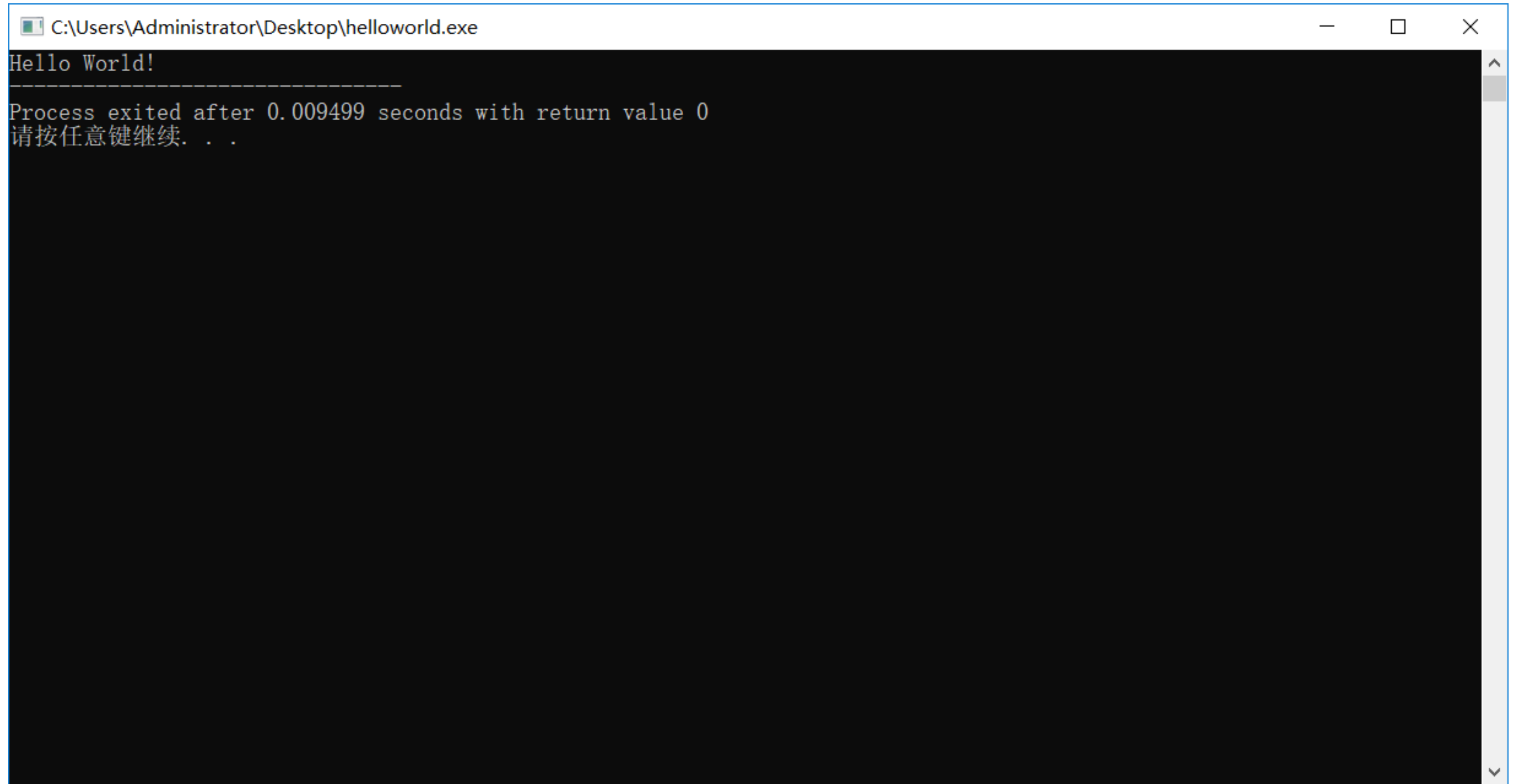
第一个C语言程序（四）

- 按快捷键 <F11> 或点击左上角 “Excute/Compile or Run” 编译并运行程序（注意以下红色区域的内容，必须 Errors 和 Warnings 都是 0）



第一个C语言程序（五）

□ 运行结果



A screenshot of a Windows command prompt window titled "C:\Users\Administrator\Desktop\helloworld.exe". The window has a black background and white text. The output of the program is as follows:

```
Hello World!
-----
Process exited after 0.009499 seconds with return value 0
请按任意键继续. . .
```

示例解释

□ helloworld.c代码虽短，但五脏俱全，我们解析一下该代码

```
1  /* This is my first C source file*/
2  #include<stdio.h>
3  □ int main() {
4      printf("Hello World!");
5      return 0;
6  }
```

C语言程序的基本语法：注释

```
/* This is my first C source file*/
```

- 首先，`/**/`括起来的部分是**注释**
 - 注释在代码中**并不起到实际的作用**——仅是为了方便阅读代码而做的注解
- 注释可以有多种写法

```
/* 一条注释 */
```

```
/* 一条注释  
另一条注释 */
```

```
/*  
一条注释  
*/
```

```
// 一条注释
```

C语言程序的基本语法：头文件

```
#include<stdio.h>
```

- ❑ `#include<>`括起来的部分是**头文件**
- ❑ 不同头文件包含了不同的预先写好的代码
 - 比如 helloworld.c 中的头文件 `stdio.h`（标准输入输出头文件）即包含了下面将会用到的 `printf()`
- ❑ `#include`是**预处理**指令：把头文件中内容复制到该位置
 - 预处理器会在编译器之前接手处理这条指令

C语言程序的基本语法：main函数

```
int main() {  
    printf("Hello World!");  
    return 0;  
}
```

↑
main函数主体

- ❑ main函数是整份代码的入口，C程序全部从main开始执行
- ❑ int是函数的返回类型，表示main函数执行结束后，会向操作系统返回一个整数
- ❑ 花括号内部将定义main函数具体的实现

C语言程序的基本语法：main函数

```
printf("Hello World!");
```

- printf也是一个函数，在stdio.h头文件中定义的输出函数
 - 该函数可以将有用的信息打印到标准输出中
 - 这里打印的是：Hello World!
- 尝试修改此处，看下是否能打印出其他的信息
 - 多写几行输出命令，能否打印如下图形？

```
  *  
 ***  
*****
```

C语言程序的基本语法：return

```
return 0;
```

- ❑ return通常意味这一个函数的结束
- ❑ 正如其名字所表示的，return将返回紧随在它后面的表达式
 - 这里return的表达式是0，也就是说main函数返回了0
- ❑ 返回给谁呢？返回给了操作系统，告诉计算机这份代码成功执行了
- ❑ 如果main函数返回的不是0，那么计算机就会认为此代码没有成功执行，产生了错误（Error）

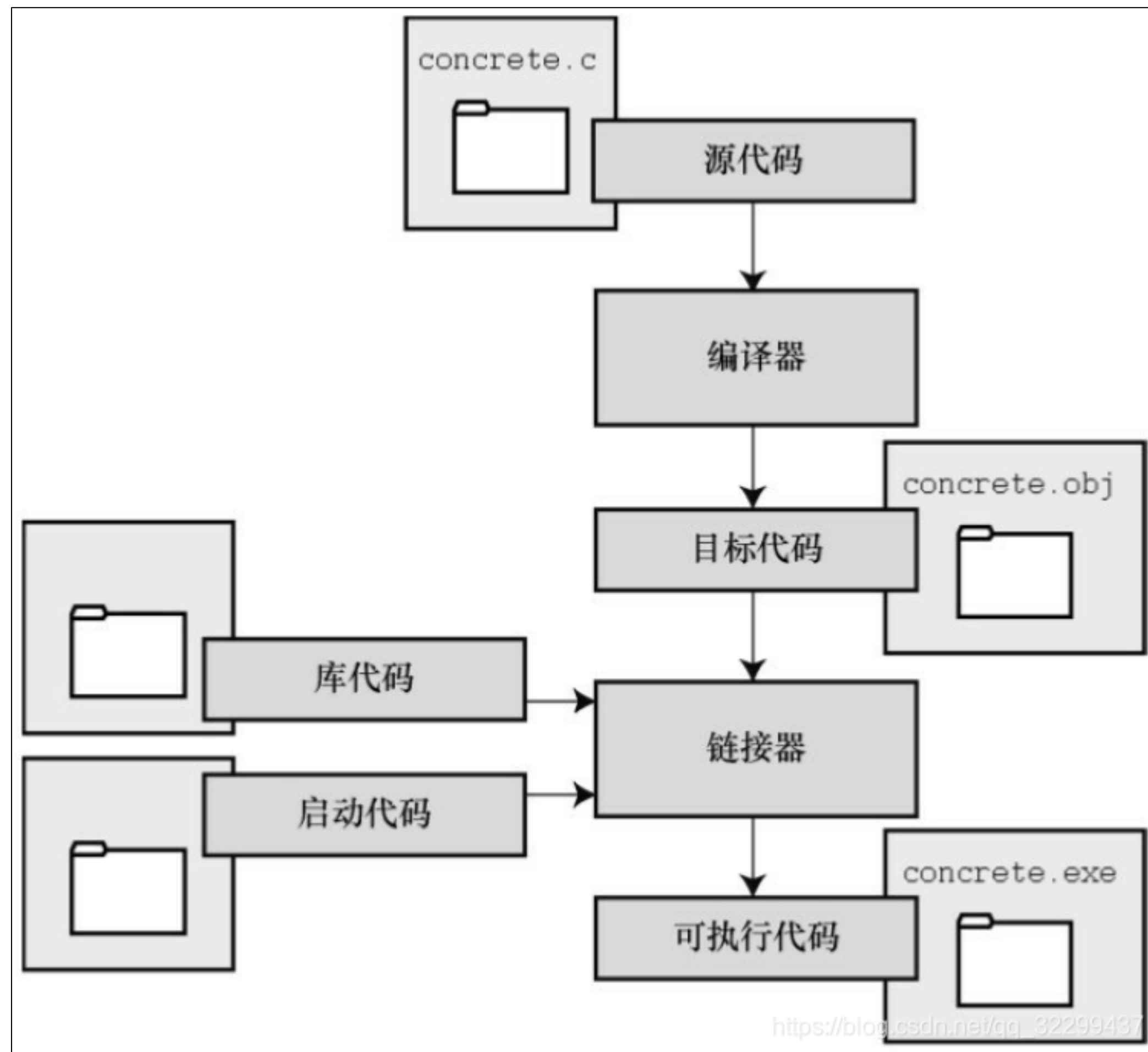
C语言程序的基本语法：分号

- 细心的同学可能已经发现了，main函数的两行代码都具有分号 “;”
- 分号在C语言中实际上与汉语的句号是一致的，它表明了一个代码语句的结束（注意是半角分号 “;” 而不是全角分号 “; ”）

C程序的基本结构

预处理 → `#include<stdio.h>`
函数头 → `int main(void)`
函数体 → `{`
`printf("Hello World!\n");`
`return 0;`
`}`

编译过程

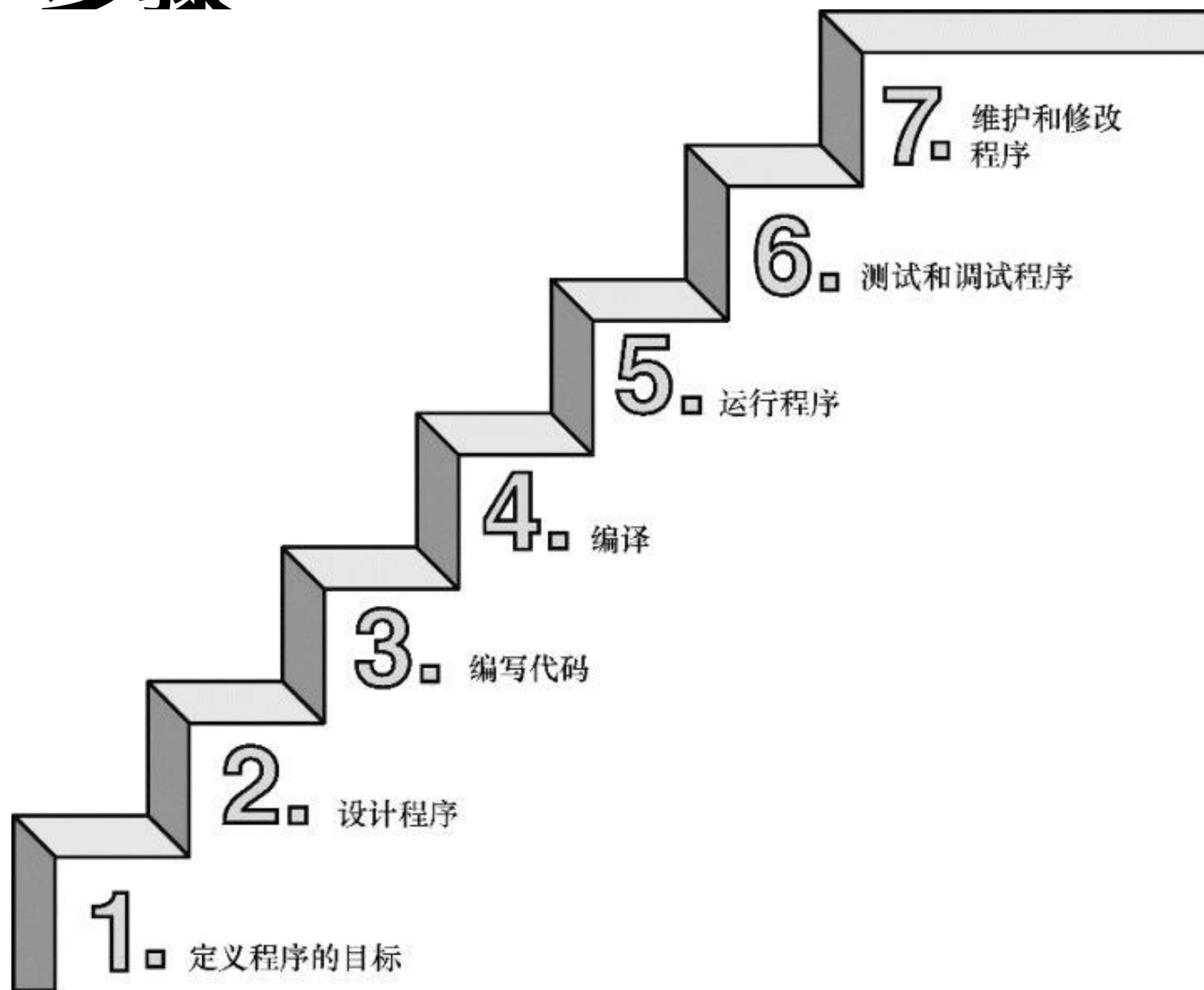


程序可读性

- ❑ 选择有意义的函数名与变量标识符，如用 feet 而非 x
- ❑ 适当注释
- ❑ 适当空行

```
int main(void) /* 把2英寻（测水深的单位）转换成英尺*/  
  
{  
    int feet, fathoms;      使用有意义的变量名  
    使用空行  
    fathoms=2;  
    feet=6*fathoms;         每行一条语句  
    printf("There are %d feet in %d fathoms!\n", feet, fathoms);  
    return 0;  
}
```

编程的7个步骤



调试程序

□ 以下程序有什么问题？编译试试看？

```
#include<stdio.h>
int main(void)
{
    printf("Hello World!")
    return 0;
}
```

□ 尝试调试程序

总结

- 编程语言是用来编写计算机程序的标准化语言
- C语言是强大、简洁的高级编程语言，具备很好的移植性
- C程序由一个或多个C函数组成，必须包含main函数
- printf()可以用于输出想要表达的内容

阅读

- 《C Primer Plus》第1-2章

谢谢大家！

邮箱: yuesh5@mail.sysu.edu.cn